

PRE-CLASS READING 2 OF 5

Designing Machine Learning Systems — Ch. 1-2

Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media. Chapters 1-2.

ASSIGNED READING

Ch. 1 — Overview of Machine Learning Systems ~40 min

Ch. 2 — Introduction to ML Systems Design ~45 min

If short on time, read this summary in full and skim the two chapters —

» ~85 min for both chapters • this (detailed) summary ~15 min

Note on currency. The book is from 2022, so its running examples are classic ML and early transformers (BERT), not today's LLMs. Read 'ML system' as 'AI system' throughout — the systems framework (objectives → requirements → lifecycle) is exactly the lens Class 2 uses for foundation-model and RAG applications, and it has aged well. This is a detailed summary because many of you won't have the book; it is still a map, not the territory.

KEY TAKEAWAY

An AI system is not the algorithm — it is the whole system: the business reason it exists, the data stack, the infrastructure, deployment, monitoring, and the people who own each part. Two ideas run through both chapters: (1) tie your model's metrics to a business metric or the project dies, and (2) a good system is judged by four requirements — reliability, scalability, maintainability, adaptability — not by benchmark accuracy. Building AI is an iterative, never-ending loop, not a one-off.

WHY IT MATTERS FOR THIS COURSE

This is the engineering backbone of Class 2. Chip Huyen's 'the algorithm is a small part of the system' is precisely the picture capsule C1 draws — compute, data, serving, monitoring, people. Her research-vs-production contrast (fast inference and low latency, messy shifting data, fairness, interpretability) is the substance of C2's build-vs-buy trade-offs and the economics thread. 'ML systems fail silently' and 'tie ML metrics to business metrics' are the reason C4 treats evals as a stack layer. And she names the idea directly — ML as 'Software 2.0' (Karpthy, Reading 3): a program learned from data, as opposed to the hand-coded rules or lookup tables of Software 1.0. That lens fits Class 1's contrast too — both the scikit-learn classifier and the foundation-model API are Software 2.0 (small vs large); a plain lookup table would be the 1.0 counterpart.

Feeds Class 2: C1 (map the stack) • C2 (build-vs-buy: research vs production) • C4 (evals) • the lifecycle thread.

MAIN POINTS

Chapter 1 — the system is not the algorithm

- **What an AI system actually is.** The model is a small piece. The system also includes the business requirement that started it, the interface, the data stack, and the logic + infrastructure to develop, deploy, monitor and update models. MLOps = the tools and practices for bringing models into production; systems design looks at all of it holistically.

Chapter 1 — when (and when not) to use ML

- **The working definition, as five conditions.** ML learns complex patterns from existing data to make predictions on unseen data. Each word is a test: (1) the system can learn; (2) a complex pattern exists — if it's simple, use a rule or lookup table; (3) data exists or can be collected (zero-shot still relied on prior data; you can launch data-less via continual learning but it's risky; hence the 'fake-it-till-you-make-it' launch); (4) it's a prediction problem (even compute-heavy tasks get reframed as 'predict the output'); (5) unseen data shares the training distribution.
- **It helps most when** the task is repetitive, wrong predictions are cheap, it runs at scale, and the patterns keep changing (hand-written rules go stale — spam is the classic).
- **Don't use ML when** it's unethical, a simpler solution works, or it isn't cost-effective. If the whole problem isn't ML-shaped, carve off the part that is (e.g. match a query to an FAQ, else route to a human).

Chapter 1 — ML use cases (industry examples)

- **Consumer vs enterprise.** Consumer apps are easier to distribute but harder to monetise and more latency-sensitive; enterprise apps tend to demand higher accuracy but tolerate more latency. Most ML value is in the enterprise.
- **Consumer examples.** search, recommenders, predictive typing, photo enhancement, face/fingerprint unlock, machine translation, smart assistants, home monitoring.

- **Enterprise examples.** fraud detection (anomaly detection on transactions), price optimisation (dynamic pricing for ads, flights, ride-share), demand forecasting (inventory), lowering customer-acquisition cost (targeting, timed discounts), churn prediction (customers and employees), support-ticket classification/routing, brand monitoring + sentiment analysis, and healthcare (skin-cancer detection, diabetes diagnosis — usually via a provider, given accuracy and privacy).

Chapter 1 — research vs production (five differences)

- **Different stakeholders, conflicting objectives.** Research chases state-of-the-art on one benchmark; production juggles many owners. Huyen's restaurant-recommender: ML engineers want accuracy, sales want expensive restaurants (higher fees), product wants <100 ms latency, the platform team wants stability, the manager wants margin — no single model satisfies all.
- **Compute priority flips.** Research prizes fast training + high throughput; production prizes fast inference + low latency. Batching can raise latency and throughput together. Latency is a distribution, not an average — reason in percentiles (p50/p90/p99) and watch the tail (Amazon's slowest requests are often its most valuable customers). Latency costs money: ~100 ms → -7% conversion (Akamai); +30% latency → -0.5% conversion (Booking.com); a 3 s load loses half of mobile users (Google).
- **Data differs.** Research data is clean, static, benchmarked; production data is messy, shifting, biased, privacy-laden, and part-historical, part-live.
- **Fairness and interpretability become requirements.** Optional in a paper, mandatory in production — biased lending/résumé/credit scoring show the cost; Hinton's 'black-box surgeon' thought experiment frames the interpretability tension.
- **Two trade-off asides.** Ensembling wins competitions but is often too complex/slow to ship; and leaderboards can mislead (multiple-hypothesis testing, and the hard prep is already done for you).

Chapter 1 — AI systems vs traditional software

- **Code + data + artifacts, entangled.** Traditional software separates code from data; AI fuses them — so you must version and test data as well as code. Not all data is equal (a rare cancerous-lung scan is worth more than another normal one), and indiscriminately ingesting data can hurt the model and invite data-poisoning.
- **Size and monitoring.** Models with hundreds of millions to billions of parameters need gigabytes of RAM, are hard to serve (especially at the edge), and degrade quietly. BERT-large (340M params, 1.35 GB) went from 'too big to be practical' to used in almost every Google English search within ~2 years.

Chapter 2 — business vs ML objectives

- **Tie ML metrics to business metrics.** Companies don't care about 94%→94.2% unless it moves a business number; projects that chase ML metrics get killed. Map performance to what the business counts — ad revenue, fraud dollars stopped. Netflix's 'take-rate' links recommendations to streaming hours and lower cancellations.
- **Attribution is hard; hype is real.** Sometimes ML is one component in a chain (e.g. a cybersecurity pipeline) and its effect is hard to isolate. Beware 'AI-washing' — 'AI-powered' used purely as marketing.
- **Returns compound with maturity.** Per an Algorithmia survey, ~75% of mature adopters deploy a model in under 30 days, vs ~60% of beginners taking longer. 'Magically: possible. Overnight: no.'

Chapter 2 — the four requirements (the core framework)

- **Reliability.** Keep performing correctly under faults — but 'correct' is hard for ML, which fails silently (a wrong prediction looks like a right one). This is why you need evals.
- **Scalability.** Systems grow in complexity, in traffic, and in model count (one startup ran 8,000 models for 8,000 customers). Handle it with resource scaling — autoscaling up/down (Amazon's autoscaling failure on Prime Day was estimated at \$72–99M/hr) — and artifact management (managing 100 models ≠ managing 1; monitoring and retraining must be automated).
- **Maintainability.** Many contributors with different tools; version code, data and artifacts, keep models reproducible, and let people collaborate without finger-pointing.
- **Adaptability.** Data and requirements shift, so the system must discover improvements and update without service interruption.

Chapter 2 — the iterative lifecycle (a loop, not a line)

- **Six steps.** 1) project scoping, 2) data engineering, 3) model development, 4) deployment, 5) monitoring + continual learning, 6) business analysis — then back to scoping.
- **Why it's a loop.** Huyen's ad-serving example bounces around: pick a metric → collect + label → engineer features → train → error analysis finds bad labels → relabel → retrain → discover the data is 99.99% negatives → collect rarer positives → retrain → the model goes stale on new data → update → deploy → revenue drops because impressions ≠ clicks → change the objective to click-through → start over.

Chapter 2 — framing the problem (task taxonomy + trade-offs)

- **Task types.** Classification vs regression (interchangeable by bucketing or thresholding); binary vs multiclass; high-cardinality (1,000+ classes needs ~100 examples each → a data-collection problem; hierarchical classification helps); multiclass vs multilabel — multilabel is the trickiest (label ambiguity, and hard to turn raw probabilities into a prediction).
- **Framing changes difficulty.** Predicting 'the next app a user opens' as a regression over (user, context, app) features scales to new apps with no retraining; framed as a classification over N apps, you must retrain whenever N changes.
- **Objective (loss) functions.** RMSE or MAE for regression, logistic/log loss for binary, cross-entropy for multiclass — chosen to match the task, not the business goal.
- **Decoupling objectives (a framework).** A newsfeed must rank by quality and engagement while filtering spam/NSFW/misinformation — goals that conflict. Option A: one combined loss (α -quality + β -engagement) — simple, but you retrain to re-tune. Option B: train a model per objective and combine their scores (α -quality + β -engagement) — re-tune without retraining, and maintain each on its own schedule (spam shifts faster than 'quality'). Pareto optimisation handles multiple objectives formally.

Chapter 2 — mind vs data

- **The debate.** Clever architecture / inductive bias ('mind') vs data + computation. Mind camp: Judea Pearl ('data is profoundly dumb'), Chris Manning (structure lets you learn more from less). Data camp: Rich Sutton's 'Bitter Lesson' (general methods that leverage computation win), Peter Norvig ('We just have more data'), and Monica Rogati's data-science hierarchy of needs (data is the foundation).
- **The trend, and the caveat.** Datasets exploded — ~0.8B tokens (2013) → GPT-2 ~10B → GPT-3 ~500B — but more low-quality data can hurt. Quality, not just quantity, is the lever.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall.

1. Huyen's test for using ML has conditions: a complex pattern exists, data is available, wrong predictions are cheap, and it runs at scale. Pick a task in your work — does it pass? Which condition is weakest?
2. Research optimises one metric; production juggles many stakeholders. For an AI feature you'd build, name two stakeholders whose objectives would conflict — and how you'd decide between them.
3. She says ML systems 'fail silently.' Describe a silent failure in your domain: what would go wrong, and what single check (an eval) would have caught it?

SOURCE LINKS

Book (O'Reilly, ISBN 9781098107963) •

<https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/>

Author • <https://huyenchip.com/>

Companion • Karpathy, 'Software 2.0' (Class 2 Reading 3) — the idea Huyen cites in Ch. 1

CAVEATS

- 1. This is a detailed summary of two chapters from a copyrighted O'Reilly book, provided because many students won't have access. It is a map, not the territory — if you can access the book, read the chapters; they are the authority.
- 2. This summary was generated and fact-checked using Anthropic models against the chapter text. It paraphrases; it does not reproduce the book. Flag anything that looks wrong.